

Compiladores: Panorama General y Contexto

Contexto Histórico y Evolución

M.C. MELISSA BRIGITTHE HINOJOSA CABELLO

Universidad Nacional Autónoma de México

Facultad de Ingeniería

20 de Agosto de 2026





Tarea 1: Historia de los lenguajes y traductores

→ Investigar acerca de:

- Primeros programas y la necesidad de traductores
- Aparición de los lenguajes de alto nivel
- Evolución de los lenguajes y su impacto
- Evolución de los compiladores

→ Elaborar un reporte con un resumen de los puntos anteriores.

→ Entrega: **Jueves 20/Agosto/2026** (antes de la clase).

- Enviar reporte (digital) por correo.
- Discusión en clase.



En los primeros años de la computación, los programas se escribían directamente como **secuencias de bits** o instrucciones muy básicas, donde cada programa estaba destinado para una máquina específica.

Repaso: ¿Cuáles eran los problemas de hacer esto?

- ▶ Cambiar de hardware implicaba reescribir cada programa por completo.
- ▶ Los errores eran extremadamente difíciles de detectar.
- ▶ El desarrollo de software era muy lento, costoso y propenso a fallos.

De aquí **surge la necesidad de** contar con **traductores** que hicieran viable la programación.

Primeros programas



El **lenguaje ensamblador** surge como el primer intento de abstracción del hardware, donde las cadenas de bits son reemplazadas por nombres simbólicos para instrucciones y direcciones.

```
home > mhinijosac > Downloads > - HelloWorld-Linux.asm
1 section .data
2     msg db "Hello, World!", 10
3     len equ $ - msg
4
5 section .text
6     global _start
7
8     start:
9         mov rax, 1      ; syscall: write
10        mov rdi, 1      ; stdout
11        mov rsi, msg     ; dirección del mensaje
12        mov rdx, len     ; longitud
13        syscall
14
15        mov rax, 60      ; syscall: exit
16        xor rdi, rdi
17        syscall
18
- HelloWorld-Mac.asm X
home > mhinijosac > Downloads > - HelloWorld-Mac.asm
1 global _main
2 extern _printf
3
4 section .data
5     msg db "Hello, World!", 10, 0
6
7 section .text
8     main:
9         lea rdi, [rel msg] ; primer argumento: string
10        xor eax, eax       ; requerido por ABI (variadic)
11        call _printf
12
13        xor eax, eax
14        ret

home > mhinijosac > Downloads > - HelloWorld-Windows.asm
1 extrn ExitProcess : proc
2 extrn GetStdHandle : proc
3 extrn WriteConsoleA : proc
4
5 .data
6     msg db "Hello, World!", 13, 10
7     len equ $ - msg
8
9 .code
10 main proc
11     sub rsp, 28h          ; shadow space (Win64 ABI)
12
13     mov rcx, -11          ; STD_OUTPUT_HANDLE
14     call GetStdHandle     ; obtiene handle de stdout
15
16     mov rcx, rax          ; handle
17     lea rdx, msg          ; dirección del mensaje
18     mov r8, len           ; longitud
19     xor r9, r9            ; chars written (NULL)
20     sub rsp, 20h          ; espacio extra
21     call WriteConsoleA
22     add rsp, 20h
23
24     xor rcx, rcx
25     call ExitProcess
26 main endp
27 end
28
```

Aparición de los lenguajes de alto nivel



El lenguaje ensamblador hace posible la programación, mas no la hace escalable. Por ello, surgen los **lenguajes de alto nivel** cuando programar en ensamblador deja de ser viable para escenarios con sistemas grandes.

Los lenguajes de alto nivel están diseñados para **expresar algoritmos y estructuras lógicas** de forma independiente del hardware, por lo que su ejecución siempre depende de un traductor (compilador, intérprete o sistema híbrido).

Ventajas:

- ▶ Se utilizan abstracciones cercanas al razonamiento humano.
- ▶ Ya no es necesario conocer registros, direcciones de memoria o instrucciones específicas de la CPU.
- ▶ El código es más legible, portable, y fácil de mantener.
- ▶ Se introduce una separación clara entre **qué hace** el programa y **cómo lo ejecuta** la máquina.

Evolución de los lenguajes y su impacto



Los primeros lenguajes de alto nivel surgen **para atender propósitos específicos**, destacando aquellos para el ámbito científico y aplicaciones de negocio. Esas nuevas necesidades impulsaron a su vez la evolución de los compiladores, añadiendo nuevas capacidades o funciones.

- **FORTRAN.** Creado por IBM en 1957, surge específicamente para cómputo científico, numérico y de ingeniería, con optimización como su objetivo central.

```
program suma
  implicit none
  integer :: i
  real :: suma
  real, dimension(5) :: datos = (/ 1.0, 2.0, 3.0, 4.0, 5.0 /)

  suma = 0.0
  do i = 1, 5
    suma = suma + datos(i)
  end do

  print *, "Suma =", suma
end program suma
```

Evolución de los lenguajes y su impacto



Los primeros lenguajes de alto nivel surgen **para atender propósitos específicos**, destacando aquellos para el ámbito científico y aplicaciones de negocio. Esas nuevas necesidades impulsaron a su vez la evolución de los compiladores, añadiendo nuevas capacidades o funciones.

- **LISP.** Creado en 1958 para procesamiento simbólico y desarrollo de inteligencia artificial; introduce un nuevo paradigma funcional y conceptos como memoria dinámica, evaluación en tiempo de ejecución y estructuras simbólicas basadas en listas.

```
(setq datos '(1 2 3 4 5))  
(setq suma 0)  
  
(dolist (x datos)  
  (setq suma (+ suma x)))  
  
(print suma)
```

Evolución de los lenguajes y su impacto



Los primeros lenguajes de alto nivel surgen **para atender propósitos específicos**, destacando aquellos para el ámbito científico y aplicaciones de negocio. Esas nuevas necesidades impulsaron a su vez la evolución de los compiladores, añadiendo nuevas capacidades o funciones.

- **COBOL.** Creado en 1959 para aplicaciones comerciales y procesamiento de datos empresariales; introduce estructuras de datos complejas y separación clara entre datos y lógica.

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. SUMA-ARRAY.
```

```
DATA DIVISION.  
WORKING-STORAGE SECTION.
```

```
01 DATOS.  
    05 DATO PIC 9 OCCURS 5 TIMES  
        VALUE 1 2 3 4 5.
```

```
01 SUMA      PIC 99 VALUE 0.  
01 I         PIC 9  VALUE 1.
```

```
PROCEDURE DIVISION.  
    PERFORM VARYING I FROM 1 BY 1 UNTIL I > 5  
        ADD DATO(I) TO SUMA  
    END-PERFORM
```

```
DISPLAY "SUMA = " SUMA  
STOP RUN.
```

Evolución de los lenguajes y su impacto



Los primeros lenguajes de alto nivel surgen **para atender propósitos específicos**, destacando aquellos para el ámbito científico y aplicaciones de negocio. Esas nuevas necesidades impulsaron a su vez la evolución de los compiladores, añadiendo nuevas capacidades o funciones.

- **C.** Creado entre 1969 – 1972 como un lenguaje de propósito general (aunque centrado en desarrollo de sistemas operativos y control del hardware), introduce un equilibrio clave entre abstracción y control, sirviendo como base para muchos lenguajes posteriores.

```
#include <stdio.h>

int main() {
    int datos[] = {1, 2, 3, 4, 5};
    int suma = 0;

    for (int i = 0; i < 5; i++) {
        suma += datos[i];
    }

    printf("Suma = %d\n", suma);
    return 0;
}
```

Evolución de los lenguajes y su impacto



A partir de la década de 1990, la motivación para el desarrollo de nuevos lenguajes ya no se relacionó con el hardware, sino con **nuevos contextos de software**: sistemas distribuidos, seguridad informática, mayor productividad o despliegue multiplataforma, entre otros.

Cada lenguaje moderno (de alto nivel) surge para resolver un problema específico:

- ▶ **Java** prioriza portabilidad y ejecución en múltiples plataformas.
- ▶ **Python** se centra en productividad y rapidez de desarrollo.
- ▶ **Rust** prioriza confiabilidad y seguridad de memoria en compilación.
- ▶ **Go** busca simplicidad y buen desempeño en sistemas concurrentes.

—> Entonces, los compiladores modernos no solo traducen código, sino que verifican, optimizan y hacen cumplir las garantías para las cuales cada lenguaje se ha diseñado.



FORTRAN es el primer lenguaje de alto nivel ampliamente exitoso, y con él surge el primer compilador en sentido moderno, conocido como ***FORTRAN Compiler*** o ***FORTRAN I***.

¿Por qué se considera el primero?

- ▶ Traducía un lenguaje de alto nivel real.
- ▶ Generaba código eficiente, comparable al ensamblador.
- ▶ Introdujo análisis léxico y sintáctico implícitos, así como optimización de código.

—> Antes de FORTRAN existían ensambladores y traductores simples, pero **no compiladores de alto nivel**.



La evolución de los compiladores sigue la misma lógica que la de los lenguajes: **resolver nuevas restricciones técnicas**. Inicialmente, la compilación era exclusivamente estática; posteriormente, surge la compilación dinámica.

Compilación estática:

- ▶ Traducción previa a ejecución.
- ▶ Alto rendimiento.
- ▶ Binarios dependientes de la plataforma.
- ▶ Poca flexibilidad ante cambios de plataforma o comportamiento.
- ▶ Ejemplos: C, C++, Rust.

Compilación dinámica:

- ▶ Traducción en tiempo de ejecución.
- ▶ Menor rendimiento inicial.
- ▶ No se genera un binario nativo final antes de ejecutar.
- ▶ Mayor flexibilidad.
- ▶ Ejemplos: Python, JavaScript.



Para abordar el compromiso entre ejecución flexible y eficiencia en tiempo de ejecución, aparece la compilación **Just-In-Time (JIT)**. JIT combina portabilidad y rendimiento al identificar las secciones críticas de un programa y aplicar optimizaciones dinámicas.

En lugar de compilar todo de antemano:

- ▶ El código se ejecuta inicialmente de forma interpretada o semi-interpretada.
- ▶ Se observa el comportamiento real del programa.
- ▶ Se identifican las partes que se ejecutan con mayor frecuencia.
- ▶ Solo esas partes se compilan a código nativo optimizado.